

Tutorium 7

Bäume & Altklausuraufgaben

Grundlagen der Programmierung

15. Juni 2026

Bäume

Altklausur & Blatt 4

Praxis

Binärbäume in Haskell

Bäume sind rekursive Datenstrukturen. Ein Baum kann leer sein (`Nil`) oder ein Knoten (`Node`) mit einem Wert und zwei Teilbäumen (links und rechts).

Definition eines Binärbaums

```
1 data BinTree a = Nil | Node a (BinTree a) (BinTree a)
2   deriving (Show, Eq)
```

Beispiel eines Knotens (Blatt): `Node 3 Nil Nil`

Operationen auf Bäumen

Analog zu Listen werden Operationen auf Bäumen über **Pattern Matching** realisiert. Wir definieren einen Fall für `Nil` und einen für `Node`.

Knotensumme

```
1 sumTree :: Num a => BinTree a  
  -> a  
2 sumTree Nil = 0  
3 sumTree (Node v l r) =  
4   v + sumTree l + sumTree r
```

Knoten zählen

```
1 countNodes :: BinTree a -> Int  
2 countNodes Nil = 0  
3 countNodes (Node _ l r) =  
4   1 + countNodes l +  
      countNodes r
```

Tree Fold (Fold für Bäume)

Da die rekursiven Traversierungen immer ähnlich aussehen, können wir eine höhere Funktion `foldTree` definieren:

`foldTree`

```
1 foldTree :: (a -> b -> b -> b) -> b -> BinTree a -> b
2 foldTree _ z Nil = z
3 foldTree f z (Node v left right) =
4     f v (foldTree f z left) (foldTree f z right)
```

Damit lassen sich `sumTree` und `countNodes` extrem kurz schreiben:

```
1 sumTree' = foldTree (\v l r -> v + l + r) 0
2 countNodes' = foldTree (\_ l r -> 1 + l + r) 0
```

Aufgabe: Elemente Zählen (mit Filter vs. Fold)

Klausuraufgabe: Implementieren Sie `count`, die die Anzahl der Elemente zurückgibt, die ein Prädikat erfüllen.

Lösungswege

```
1  -- Mit Filter (einfach!)
2  count' :: (a -> Bool) -> [a] -> Int
3  count' p xs = length (filter p xs)
4
5  -- Mit Fold
6  count'' :: (a -> Bool) -> [a] -> Int
7  count'' p xs = foldr (\x acc -> if p x then acc + 1 else acc) 0 xs
```

Live Demo



01_altklausuraufgaben.hs, 02_baeume.hs, sheet04.hs

Wir programmieren Bäume und besprechen Blatt 4 sowie weitere
Klausuraufgaben!